Introduction to Object-Orientation

CCS1200
Introduction to Object-Orientation

Dr Ioanna Stamatopoulou

Prerequisites

- **No knowledge** of object-oriented this_or_that is required

But . . .

- …it is **necessary** that you refresh your knowledge on Java syntax **!!!**

Outline

At the end of this lecture you will know what the following are:

- Class
- Attribute
- Behaviour
- Method
- Object, instance of a class
- Attribute value
- Responsibility
- Message

The object-oriented way of thinking

- We tend to understand the world in terms of **objects**:
  - the **car** parked across the street
  - my **baby** at home
  - CITY's **building**
- Each object is unique but it belongs to a certain **category** of objects, it is **an instance of a class**
  - The class of cars (all cars in the world)
  - The class of babies (all babies in the world)
  - The class of buildings (all buildings in the world)

Class attributes

- All classes have **attributes** (features we use in order to describe them):
  - The **colour** of a car
  - The **hair-colour** of a baby
  - The **number of floors** of a building
- **Attributes are common** to all objects of a class
  - All cars have a colour
  - All babies have a hair-colour
  - All buildings have a number of floors

Class attributes – Object values

- Though attributes are common for all objects of a class, objects may have different **values** for each attribute
  - The blue car, the red car, the silver car
  - a blond baby, a red-haired baby
  - a 5-floor building, a 12-floor building

Class behaviour

- All objects also exhibit a **behaviour**
  - A car **accelerates**
  - A baby **cries**
  - A building **rents/sells flats/offices**
- The behaviour of a class is **common** to all objects of a class
  - All cars accelerate
  - All babies cry
  - All buildings rents/sells flats/offices

Objects, Responsibility, Messages and Methods

Solving a problem the OO way	Problem: I need a coffee
Find an object of the appropriate class	List two objects that can make coffee
Send a **message** to this object with our request	What are the messages you send to the objects listed earlier?

- It is the **responsibility** of the selected object to satisfy my request
- The selected object has a **method** (a way, an algorithm) to satisfy my request

Procedural Solution (what you know so far…)

- Go to super-market
- Buy coffee and sugar
- Go to kitchen
- Boil water
- …
- Stir contents in cup
- …

Your program is responsible for doing everything

Object-Oriented Solution (what we will learn this semester)

- Go to local coffee-shop
- Describe the coffee I want, and…
- DONE!!!!

- …because someone else is responsible for making the coffee

Classes and Instances

- All coffee-shop employees accept requests to make coffee (among other requests) and use a method to satisfy this request
  - **All** coffee-shop employees **behave the same way**
- The term CoffeeShopEmployee represents the **class** (category) of all coffee-shop employees
- Nick **is a** coffee-shop employee
  - He is an **instance of the class** CoffeeShopEmployee
  - He behaves like all coffee-shop employees

Characteristics of OOP

- Everything is an object
- Every object has a **type** – it belongs to a **class**

  type ⇔ class
- Each object has a set of attributes made up of other objects
- All objects of a particular class have the same responsibilities, can receive the same messages, and use the same method to achieve a behaviour
- **A program is a bunch of objects telling each other what to do by sending messages**

Classes

CCS1200
Introduction to Object Orientation

Dr Ioanna Stamatopoulou

Class Identification

- The first task in OOP is to identify what type(s) of objects are appropriate for our problem
  - Determine the classes
- Some of the required classes may be already implemented
  - e.g. Java's class String
- … but what happens if they are not
  - like a HotelRoom, a Student, etc.
- We have to **define the class** !

What is a Class?

- A **Class** is a means for describing the
  - **properties** (**attributes**) and
  - **behaviour** (**methods**)

  of an object in real life that some program has to deal with
- It is a description of a **category** of objects
- It represents what is common between all instances of the class (objects)

The Rectangle Class

- All rectangles belong to the class **Rectangle**
- All rectangles share:
  - the same properties
  - the same behaviour

Class Definition

- A **class** in Java is defined in terms of:
  - the **class name**
  - the **attributes** of its objects
  - the **methods** of its objects
- The body of the class definition is delimited between a left **{** and a right **}** brace

The Rectangle Class Definition

Class Name

Attributes

Methods

```
public class Rectangle{
    // attributes (describe state)
    // they form the private (hidden) part
    private double height;
    private double width;

    // methods(define behaviour)
    // they form the public (accessible) part
    public void setHeight(double height){
        this.height = height;
    }
    public boolean isSquare(){
        return height == width;
    }
    …
} //end of class definition
```

Class Attributes

- Attributes are:
  - the characteristics that we need in order to describe objects of a class
  - **the information we need to know about each of the objects**

- **An Attribute is a variable!**
  - It has a **name**
  - It has a **type**
  - It receives a **value**

Rectangle Attributes

- **For any and all rectangles** we know:
  - they have a **height**
  - they have a **width**

```
private  double height;
private  double width;
```

Attribute types

Attribute names

Class Methods

- The behaviour of an object is implemented by its **methods**
  - Methods in OOP are not static: they will be called for each one of the objects of the class, not for the class as a whole
- The behaviour of an object has partly to do with manipulating its attributes' values

Rectangle Methods

- All rectangles have common methods:
  - Their width can be changed
  - Their height can be changed
  - Their perimeter can be calculated
  - Their area can be calculated
  - …

Rectangle Methods

```
// Selectors:
// methods that don't modify the state
// but return information about the objects
public double getHeight(){
    return height;
}

public double getWidth(){…}
public boolean isSquare(){…}
```

Selector Method `getHeight`:
It does not change the value of the `height` attribute.
It only returns its value

Rectangle Methods (cont'd)

```
//Modifiers:
//methods that modify the state (attribute values)
public void setHeight(double height){
    this.height = height;
}


public void setWidth(doubleWidth){
     this.width = width;
}
```

Modifier Method `setHeight`:
It **changes** the value of the `height` attribute to the value of the parameter (argument)

The `this` keyword

- A name used for a class attribute can be reused for a variable declared in a method of the class

To distinguish between the two the keyword `this` can be used

Same Name

```
public class Rectangle{
    // attributes (describe state)
    private double height;
    …

    // methods(define behaviour)
    public void setHeight(double height){
        this.height = height;
    }
    …
} //end of class definition
```

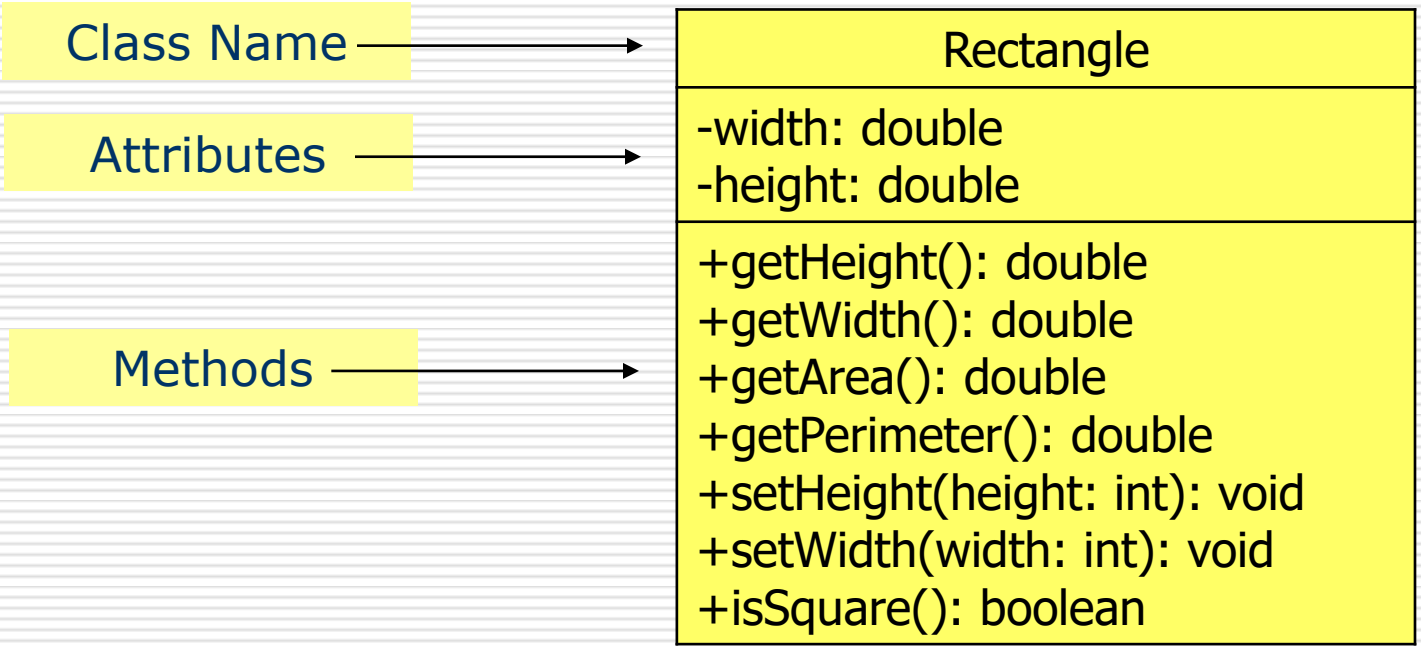
Types of methods: Modifiers

- Modifiers (write or **set** methods):
  - Any method which changes the state of an object:
    - `void setHeight()`
    - `void setWidth()`
  - … Like buttons on a machine

Types of methods: Selectors

- Selectors (read or **get** methods):
  - A method which allows access to an attribute (gives us its value) but does not change the state of an object:
    - `int getWidth()`
    - `boolean isSquare()`
  - … Like displays on a machine

UML representation of a class



The Student Class

- All Students:
  - Have a name, registration number, level of studies, and a total of credits gained the current academic year
  - Gain some (how many?) credits every time they pass a unit
  - Proceed to the next level of studies, if their total credits are 100

Classes as Abstract Data Types

Need for classes not in User Requirements

- Sometimes there is a need for more classes than the ones identified in the user requirements
- Example 1:

  Consider the HotelRoom class, and an attribute for recording the checkOut time of a customer

The Flight class

- Example 2:

  Let us assume a problem description that involves flights, where the only necessary class of objects identified is the Flight class

- Two **attributes** of the Flight class will be the departure **time** and the arrival **time** of the flight
  - However, the flight class is NOT responsible for handling time

User-defined Abstract Data Types (ADT)

- So what will be the type of the departureTime and arrivalTime attributes?
- They could be for example Strings:
  - `private String departureTime;`
  - `private String arrivalTime;`

BAD IDEA

- Strings are not appropriate for handling time:
  - If the time now is represented by the String `"11:42:51"`, how can you create a String that represents what the time is one hour later?
  - NOT PRACTICAL!

User-defined Abstract Data Types (cont'd)

- Time is an Abstract Data Type
- We need to define our own Time class
- Flight attributes **departureTime** and **arrivalTime** will be instances of this class
  - i.e. they will be **Time objects**

The Flight class

Flight

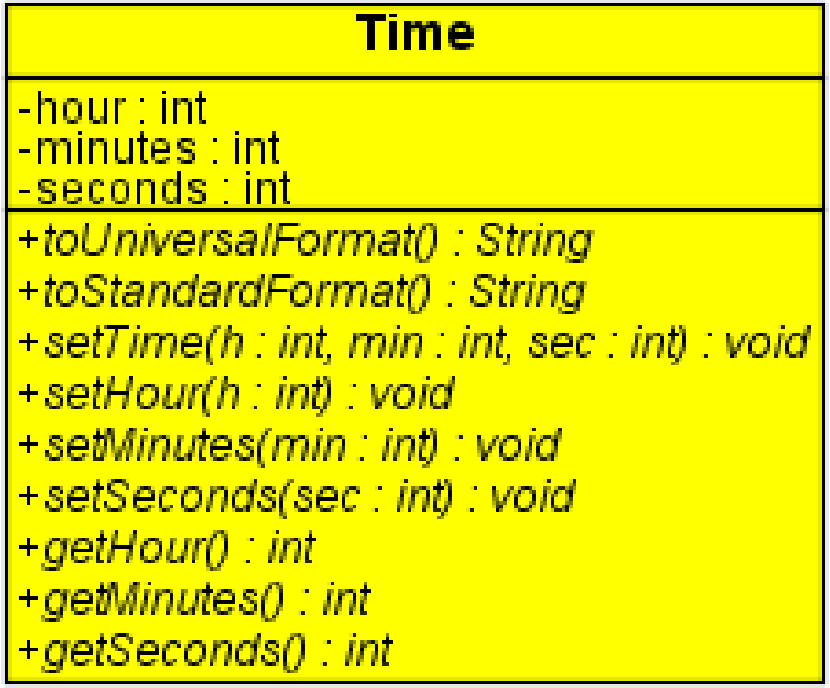
-departureAirport : String
-arrivalAirport : String
-departureTime : Time
-arrivalTime : Time

+setDepartureAirport(dep : String) : void
+setArrivalAirport(arr : String) : void
+setDepartureTime(depTime : Time) : void
+setArrivalTime(arrTime : Time) : void
+getDepartureAirport() : String
+getArrivalAirport() : String
+getDepartureTime() : Time
+getDepartureTime () : Time

The Time class

- Attributes
  - Number representing the hour
  - Number representing the minutes
  - Number representing the seconds
- Methods
  - Setting the time
  - toUniversalFormat (e.g. 23:59:59)
  - toStandardFormat (e.g. 11:59:59 pm)

Time class: UML representation



Time class Implementation: Attributes

```
public class Time{
    //integer representing the hour part
    private int hour;

    //integer representing the minutes part
    private int minutes;

    //integer representing the seconds part
    private int seconds;
}
```

Remember to declare **attributes** as **`private`**

Time class Implementation: Selector Methods

```
public class Time{
    …
    //SELECTOR (GET) METHODS
    //a method for getting the hour
    public int getHour(){
        return hour;
    }
    //a method for setting the minutes
    public int getMinutes(){
        return minutes;
    }
    //a method for setting the seconds
    public int getSeconds(){
        return seconds;
    }
    …
}//end of class
```

Notice that selector (get) methods:

1. Do not change the values of the attributes
2. (Usually) Do not accept arguments
3. Return a value (are not void)

Time class Implementation: Selector Methods (cont'd)

```
public class Time{
    …
    //ADDITIONAL SELECTOR METHODS
    //a method for returning the time in a String in
    //the universal (24-hour) format: "hh:mm:ss"
    //e.g. 17:20:50

    public String toUniversalFormat(){
        return hour + ":" + minutes + ":" + seconds;
    }
    …
}//end of class
```

Time class Implementation: Selector Methods (cont'd)

```
public class Time{
   …
   //ADDITIONAL SELECTOR METHODS (CONT'D)
   //a method for returning the time in a String in //the
   standard (12-hour) format:
   //"hh:mm:ss AM/PM" e.g. 05:20:50 PM

   public String toStandardFormat(){
      String shour, indicator;
      if (hour == 0 || hour == 12)
         shour = "12";
      else
         shour = "" + hour % 12;
      if (hour < 12)
         indicator = "AM";
      else
         indicator = "PM";
      return shour +":"+ minutes +":"+ seconds + indicator;
   }
   …
}//end of class
```

Time class Implementation: Modifier Methods

```
public class Time{
    …
    //MODIFIER (SET) METHODS
    //a method for setting the hour
    public void setHour(int h){
        hour = h;
    }
    //a method for setting the minutes
    public void setMinutes(int m){
        minutes = m;
    }
    //a method for setting the seconds
    public void setSeconds(int s){
        seconds = s;
    }
    …
}//end of class
```

Notice that modifier (set) methods:

1. Change the values of the attributes
2. (Usually) Accept arguments
3. (Usually) Do not return a value (are void)

Time class Implementation: Modifier Methods (cont'd)

```
public class Time{
    …
    //MODIFIER (SET) METHODS (CONT'D)
    //a method for setting all three attributes
    public void setTime(int h, int m, int s){
        hour = h;
        minutes = m;
        seconds = s;
    }
    …
}//end of class
```

Notice that a modifier (set) method does not necessarily change the value of only one attribute!

The Date class

- Attributes
  - Number representing the day
  - Number representing the month
  - Number representing the year
- Methods
  - Setting the date
  - toUniversalFormat (e.g. 2012/11/30)
  - toEuropeanFormat (e.g. 30 November 2012)
  - toAmericanFormat (e.g. November 30, 2012)